MEDSENTRY
AI-POWERED SMART ANTIBIOTIC & MEDICATION MANAGEMENT PLATFORM

A PROJECT REPORT

Submitted by

Subhash R (CH.SC.U4AIE23054)

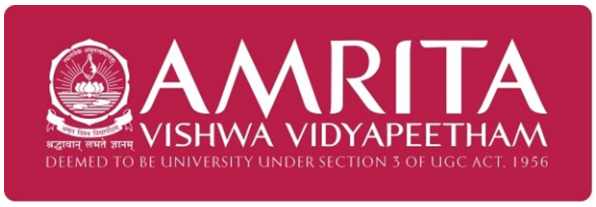
Veena Sundari A (CH.SC.U4AIE23060)

in partial fulfillment for the award of the degree of
BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

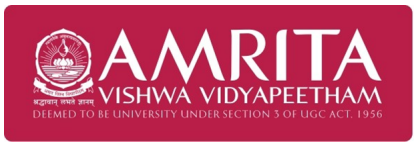
Dr. G. Bharathi Mohan

Submitted to



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
AMRITA SCHOOL OF COMPUTING
AMRITA VISHWA VIDYAPEETHAM
CHENNAI - 601103

Academic Year 2026–2027
OCTOBER 2026



This is to certify that the project report entitled **"MEDSENTRY: AI-POWERED SMART ANTIBIOTIC & MEDICATION MANAGEMENT PLATFORM"** is a bonafide work carried out by **Subhash R (CH.SC.U4AIE23054)** and **Veena Sundari A (CH.SC.U4AIE23060)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** under the course **Full Stack Development (22AIE457)** at Amrita Vishwa Vidyapeetham during the academic year 2026–2027.

SIGNATURE

Name Signature

DR. G. BHARATHI MOHAN
Assistant Professor (Sr.Gr.)
Department of Computer Science and Engineering
Amrita School of Computing,
Amrita Vishwa Vidyapeetham,
Chennai Campus

Abstract

Antibiotics are among the medicines that are most often taken wrongly: courses are stopped early, bought without a prescription, or forgotten, and each of these behaviours speeds up antimicrobial resistance (AMR). Families who look after elderly relatives or children also have no shared view of who has taken what. This project builds MedSentry, a full-stack mobile platform that addresses both problems. The front end is a React Native (Expo, TypeScript) application. The back end is Supabase: a PostgreSQL 15 database with Row Level Security on every table, database triggers, stored procedures, and three Deno edge functions. The AI agent is an `ai-router` edge function that sends requests to Google Gemini, Groq (Llama 3.3 70B) or OpenRouter in that order, adds the user's profile, allergies, active courses and 7-day adherence history to the prompt, performs a deterministic allergy check before the model is called, and applies three refusal guardrails to the output.

A prescription or pill strip is scanned with the camera, text lines are regrouped by their position, drug names are matched against a 752-drug catalogue with trigram and Levenshtein matching, and the result is shown to the user for confirmation before a tracked course is created. Family circles with owner, caregiver and viewer roles let relatives log doses for each other, and every caregiver action is written to an audit table; any member can revoke caregiver consent at any time. A trigger writes anonymised misuse events and a stored procedure publishes a regional AMR heatmap only for groups of at least five events. In an offline experiment on the catalogue, the Levenshtein matcher recovered 99.5% of drug names corrupted by two character errors, whereas the trigram matcher at its configured threshold recovered 61.4%. The three guardrails blocked all six unsafe test prompts and passed all six ordinary prompts. Large-scale user trials were not carried out, and the application's AI answers are advisory only.

Keywords: Full Stack Development, React Native, Supabase, PostgreSQL, Row Level Security, AI Agent, OCR, Antimicrobial Resistance, WHO AWaRe, Family Caregiving

Contents

1 INTRODUCTION **1**

1.1 Background . . . 1
1.2 Domain Overview . . . 1
1.3 Problem Statement . . . 1
1.4 Motivation . . . 2
1.5 Objectives . . . 2
1.6 Scope of the Project . . . 2
1.7 Target Users . . . 2

2 EXISTING SYSTEM AND DOMAIN STUDY **3**

2.1 Existing System . . . 3
2.2 Limitations of Existing System . . . 3
2.3 Existing Technologies / Approaches . . . 3
2.4 Proposed Improvement . . . 4
2.5 Novelty and Key Contribution . . . 4

3 PROPOSED SYSTEM AND REQUIREMENTS **5**

3.1 Proposed System . . . 5
3.2 Major Features . . . 5
3.3 Functional Requirements . . . 6
3.4 Non-functional Requirements . . . 6
3.5 User Interaction . . . 7
3.6 Use Cases . . . 7

4 SYSTEM DESIGN AND ARCHITECTURE **8**

4.1 Overall System Architecture . . . 8
4.2 Frontend Design . . . 9
4.3 Backend Design . . . 9
4.4 Database Design . . . 10
4.5 Chatbot / AI Agent Design . . . 11
4.6 Data Flow / Workflow . . . 12

5 IMPLEMENTATION **13**

5.1 Development Environment . . . 13
5.2 Frontend Implementation . . . 13
5.3 Backend Implementation . . . 14
5.4 Database Implementation . . . 14
5.5 Chatbot / AI Agent Implementation . . . 15
5.6 Full Stack Integration . . . 15
5.7 Important Code Snippets . . . 15

6 RESULTS AND DISCUSSION **17**

6.1 System Output . . . 17
6.2 Chatbot / AI Agent Interaction . . . 18
6.3 Functional Testing . . . 18

6.4 Results . . . 19
6.5 Discussion . . . 21
6.6 Evaluation . . . 21
7 CONCLUSION AND FUTURE ENHANCEMENT 23
7.1 Conclusion . . . 23
7.2 Major Contribution . . . 23
7.3 Limitations . . . 23
7.4 Future Enhancement . . . 23
BIBLIOGRAPHY 24
APPENDIX 24

List of Figures

4.1 Overall architecture: User → AI Agent / Frontend → Backend/API → Database and Services → Response . . . 8
4.2 Core entity–relationship diagram (blue: patient data, green: family, orange: reference and surveillance) . . . 11
4.3 Workflow from prescription capture to AMR surveillance . . . 12

6.1 Home dashboard and the prescription scanning flow . . . 17
6.2 Medication tracking and family circle screens . . . 18
6.3 Symptom journal and AI agent screens . . . 18
6.4 Administrator screens . . . 19

List of Tables

2.1 Comparison of existing applications and MedSentry . . . 4

3.1 Major modules of MedSentry . . . 5
3.2 Principal use cases . . . 7

4.1 Back-end interfaces . . . 9
4.2 Main tables and fields . . . 10

5.1 Development environment . . . 13
5.2 Request path through the stack for four actions . . . 15

6.1 Representative agent interactions . . . 19
6.2 Functional test cases and results . . . 20
6.3 Top-1 matching accuracy on 749 catalogue names under simulated OCR errors . . . 21
6.4 Evaluation measures and results . . . 22

Chapter 1

INTRODUCTION

1.1 Background

Medicines work only when they are taken correctly, and this is hardest for antibiotics. A course must be completed at the prescribed dose and for the prescribed number of days. When a patient stops after two days because the fever has gone, the surviving bacteria are the ones that tolerate the drug best, and they multiply. Buying an antibiotic over the counter for a viral cold has the same effect. Both add to antimicrobial resistance (AMR), associated with about 4.95 million deaths worldwide in 2019 [1].

Poor adherence is a separate problem: about half of the patients on long-term therapy in developed countries do not take their medicines as prescribed [2]. Missed and doubled doses are common, especially when one person manages the tablets of several relatives.

1.2 Domain Overview

MedSentry belongs to mobile health (mHealth) and, within it, to medication management and antimicrobial stewardship, which promotes the correct choice, dose, duration and necessity of antibiotic use. The WHO AWaRe classification [3] supports this by dividing antibiotics into *Access* drugs (first-choice, lower resistance), *Watch* drugs (higher resistance potential, for specific conditions) and *Reserve* drugs (kept for infections that nothing else treats). An application that knows the AWaRe tier of each antibiotic can warn the user about Watch or Reserve drugs and record, anonymously, where such use occurs.

1.3 Problem Statement

Existing reminder applications record that a pill was taken, but they do not read the prescription, do not help the patient judge whether an antibiotic is necessary, and treat each user as isolated. AMR surveillance depends mostly on hospital laboratory data and misses community misuse.

The problem addressed here is to design and implement one application that (1) captures a prescription or package with the camera and turns it into a tracked course with minimal typing; (2) reminds the patient, records doses and rewards adherence; (3) lets family members and caregivers see and, with consent, update a dependent's records; (4) gives personalised, safety-limited answers through a chatbot or AI agent that knows the user's profile; and (5) produces anonymised regional misuse statistics that cannot be traced to a person.

1.4 Motivation

The team chose this problem for three reasons. Many households manage the medicines of an elderly parent or a child with a paper note or a messaging group. The problem needs every layer of a full stack system, since a scanner without a database is only a demo and a database without access control is unsafe for health data. And the AI part carries a real risk: a model asked about a child's antibiotic dose, or a lethal dose, should refuse. The agent was therefore built into the back end, with checks that cannot be bypassed from the phone. A technological solution is required because reading handwriting, matching misspelt drug names, checking allergies, enforcing who may edit whose record and aggregating many events are not realistic to do by hand.

1.5 Objectives

The specific objectives of the project are:

1. To build a React Native front end for authentication, dashboards, medication tracking, symptom journaling and family circles.
2. To design a PostgreSQL schema with Row Level Security on every table, loaded with a catalogue of drugs, AWaRe tiers and interactions.
3. To implement a camera-based scanner with position-aware text regrouping and fuzzy drug-name matching.
4. To implement an AI agent as a serverless function with provider fallback, profile-aware prompts, an allergy pre-check and three refusal guardrails.
5. To implement family circles with three roles, consent revocation and an audit trail.
6. To implement triggers for badges and AMR misuse events and a privacy-thresholded regional heatmap, and to verify and evaluate the system with scripts and test cases.

1.6 Scope of the Project

In scope. The mobile application for patients, caregivers and administrators; the Supabase back end; the AI agent with five request modes (chatbot, suitability, symptom-to-care, visual identification, prescription extraction); and the admin dashboard.

Out of scope. MedSentry does not diagnose, prescribe or calculate paediatric doses, is not a certified medical device, and has no pharmacy or hospital integration. The on-device AI tier is a skipped placeholder. The regional heatmap uses a minimum-count threshold (k-anonymity style) and is not formal differential privacy.

1.7 Target Users

Patients who take courses of medicine; **family caregivers** of elderly parents or children; **dependents and viewers** who control who may edit their records; and **administrators** who moderate accounts and view the regional heatmap.

Chapter 2

EXISTING SYSTEM AND DOMAIN STUDY

2.1 Existing System

Three families of solutions exist around this problem. *Medication reminder applications* such as Medisafe [4] and MyTherapy [5] let a user enter medicines manually, set reminder times and, in some cases, nominate a family member who is alerted when a dose is missed. *Drug reference applications and websites* give monographs, interactions and side effects but do not know what the user is actually taking. *Antimicrobial surveillance systems*, such as the WHO Global Antimicrobial Resistance and Use Surveillance System (GLASS) [6], collect resistance and consumption data from laboratories and national reporting centres.

2.2 Limitations of Existing System

- **Manual entry:** drug names, strengths and schedules are typed from the prescription, which is slow and error prone for handwriting and small print.
- **No stewardship context:** a reminder for an antibiotic looks like one for a vitamin; the AWaRe tier and the necessity of the drug are not shown.
- **Single-user model:** family features are mostly alerts, without roles, a consent switch for the dependent, or a record of what the caregiver changed.
- **Unsafe or absent AI:** general chatbots do not know the user's allergies or medicines and may answer lethal-dose or paediatric-dose questions.
- **Delayed surveillance:** laboratory-based surveillance misses early community signals such as a rise in unprescribed purchases.

2.3 Existing Technologies / Approaches

Several technologies used in existing solutions were studied and reused. **Optical character recognition** (OCR) is available on phones through engines such as Google ML Kit [16] and through multimodal language models that can read cursive handwriting [7]. **Approximate string matching** corrects OCR errors: the Levenshtein distance [8] counts single-character edits, and the PostgreSQL `pg_trgm` extension [9] compares three-character sequences and can be indexed. **Backend-as-a-service** platforms such as Supabase [10] provide a hosted PostgreSQL database, JWT authentication, row-level security and edge functions. **Language models** are offered as hosted APIs by Google [11] and Groq [12]. **Privacy of aggregates** is handled with k-anonymity [13], which suppresses small groups, or differential privacy [14], which adds noise. **Nudging** [15] uses small rewards and prompts to improve adherence.

2.4 Proposed Improvement

Table 2.1 summarises how MedSentry responds to each limitation.

Table 2.1: Comparison of existing applications and MedSentry

Dimension	Typical existing app	MedSentry
Prescription entry	Manual typing	Camera scan, regrouped text lines, fuzzy catalogue match, confirmation screen before saving
Stewardship context	None	WHO AWaRe tier stored for each antibiotic and shown on cards and monographs
Family support	Alert to one contact	Circles with owner, caregiver and viewer roles, shared medicine cabinet, consent revocation, audit log
AI assistant	None, or generic chatbot	Server-side agent with user context, allergy pre-check and three refusal guardrails
Adherence motivation	Alarms	Database trigger awards streak badges
Surveillance	Laboratory data, delayed	Trigger-fed misuse events and a heatmap published only when a group has at least five events
Data access control	Application-level checks	Row Level Security enforced inside the database

2.5 Novelty and Key Contribution

The technologies above are available to any developer; the contribution is how they are combined in this implementation:

1. **Safety checks outside the model.** The allergy check is a plain string comparison in the edge function that runs *before* the model is called, and the three refusal guardrails run on the server on both the input and the output, so a change of provider cannot remove them.
2. **Hybrid drug matching.** OCR lines are regrouped into table rows by vertical overlap (65% of line height), and candidates are matched first by a database trigram function and then by a client-side Levenshtein score of at least 70%. Section 6.4 measures both on the project catalogue.
3. **Consent-aware caregiver access in SQL.** Caregiver rights are enforced in `SECURITY DEFINER` functions used by RLS policies. A dependent can switch off caregiver edit access with one flag (`caregiver_consent_revoked`), and every caregiver action is logged to `caregiver_audit_log`.
4. **One data model for care and public health.** The `medication_courses` table that powers the patient's list also feeds, through a trigger, the anonymised `misuse_events` table. `get_amr_heatmap()` publishes only regions with at least five events in 30 days and flags a region as elevated when its last-7-day rate is at least 2.5 times its baseline rate.
5. **Provider fallback with logging.** Every call to every AI provider is logged with latency and outcome, which feeds the admin metrics screen.

Chapter 3

PROPOSED SYSTEM AND REQUIREMENTS

3.1 Proposed System

MedSentry is a mobile application backed by a managed cloud back end. A user signs up, optionally fills in a clinical profile (age, weight, height, conditions, allergies) and adds medicines by scanning a prescription or strip, by searching the 752-drug catalogue, or by entering a custom name. Each medicine becomes a *course* with a schedule and a count of remaining pills; doses are logged with one tap. The user can ask the AI agent about a medicine, scan an unknown strip or chat about the routine. Family members join a circle with a six-character invite code, and administrators use separate screens to manage users and see regional statistics.

The system solves the problem of Section 1.3 as follows: scanning removes manual entry; AWaRe tiers and interaction data bring stewardship context into the record; circles and RBAC give caregivers safe visibility; the agent answers within guardrails; and triggers convert the same records into anonymised surveillance data.

3.2 Major Features

The application is organised into the modules listed in Table 3.1.

Table 3.1: Major modules of MedSentry

Module	**Description**
Authentication	Email and password sign-up and sign-in, email verification, persistent session
Home dashboard	Active courses, doses taken today, adherence, next dose, one-tap logging
Prescription scanner	Camera or gallery capture, OCR, line clustering, catalogue matching, verification screen
Catalogue and tracker	Typeahead search, AWaRe tier, monograph, concurrent courses, course history
AI consultant and chatbot	Image scan, suitability, symptom-to-care, multi-turn chat with user context
Symptom journal	Severity 1–10, onset, body-system tags, photo, link to a course
Family circle	Invite code, three roles, member profiles, proxy dose logging, consent switch, shared cabinet
Admin portal and profile	User moderation, audit log, API usage, AMR heatmap; clinical profile, BMI, Medical ID, badges

3.3 Functional Requirements

FR1 Register, authenticate and sign out users, keeping the session across restarts.

FR2 Extract drug, dosage, frequency and duration from a scanned prescription and let the user edit them before saving.

FR3 Match drug names to the catalogue despite OCR errors, with a custom-drug fallback.

FR4 Store courses, calculate days remaining and record doses as taken, skipped or snoozed.

FR5 Award badges automatically and record symptoms linked to courses.

FR6 Create or join family circles and enforce owner, caregiver and viewer roles.

FR7 Let a member revoke caregiver consent, and log every caregiver action.

FR8 The AI agent shall use the user's profile, allergies, courses, adherence and symptoms.

FR9 The AI agent shall refuse paediatric dosing, overdose and severe self-diagnosis questions.

FR10 Record antibiotic misuse events and publish a regional heatmap only for groups of at least five events.

FR11 Let administrators manage users and circles with a mandatory reason, auditing every action.

3.4 Non-functional Requirements

Performance. A GIN trigram index supports catalogue search on the 752-row table. AI requests have a 20 s (Gemini) or 15 s (other providers) timeout, after which the next provider is tried.

Security. Every table has Row Level Security enabled. The mobile client holds only the public anonymous key; provider keys and the service-role key live in Supabase secrets. Privileged actions run only in edge functions that first verify that the caller is an administrator.

Usability. A calm colour palette, large touch targets and a bottom tab bar are used, with confirmation screens where an error would be costly (OCR results, destructive admin actions).

Reliability and scalability. The AI router falls back across providers and logs every attempt; foreign keys use `ON DELETE CASCADE` or `SET NULL` so that deleting a user leaves no orphan rows. The back end is managed PostgreSQL and serverless functions that scale by platform, and a synthetic load script inserts 500 courses and 1,000 audit rows to exercise list screens.

3.5 User Interaction

Users interact with the AI agent through two screens of the *AI* tab. The **Image Scan** screen sends a photo of a strip or prescription (as base64) to the agent in visual-identification or prescription-extraction mode. The **AI Chat** screen sends the running message list in chatbot mode. In both cases the agent looks up the user's data on the server, so the user never repeats allergies or current medicines. The response carries the text, the provider used, and whether a guardrail fired; quick-reply chips such as "Take with food?" reduce typing.

3.6 Use Cases

Table 3.2 lists the main use cases and the interactions between actor and system.

Table 3.2: Principal use cases

ID	Use case	Actor	Main flow
UC1	Add course by scanning	Patient	Capture prescription, system extracts and matches, patient edits and confirms, course saved
UC3	Ask the AI agent	Patient	Send question; server adds context and allergy warning; provider answers; guardrails filter
UC4	Log dose for dependent	Caregiver	Open member profile, tap Log Dose; RLS checks role and consent; dose and audit rows written
UC6	View AMR heatmap	Administrator	Only regions with at least five events are returned

Chapter 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall System Architecture

MedSentry has four layers (Figure 4.1). The *user* works with the *React Native application*, which includes the AI agent screens. The application talks to a *Supabase back end* through three channels: the Auth service for sign-in and tokens, the automatic REST interface to the database (PostgREST), and edge functions for logic that must run on a server. Behind the back end are the *data and external services*: the PostgreSQL database, the AI providers, and an email provider. A response travels back along the same path.

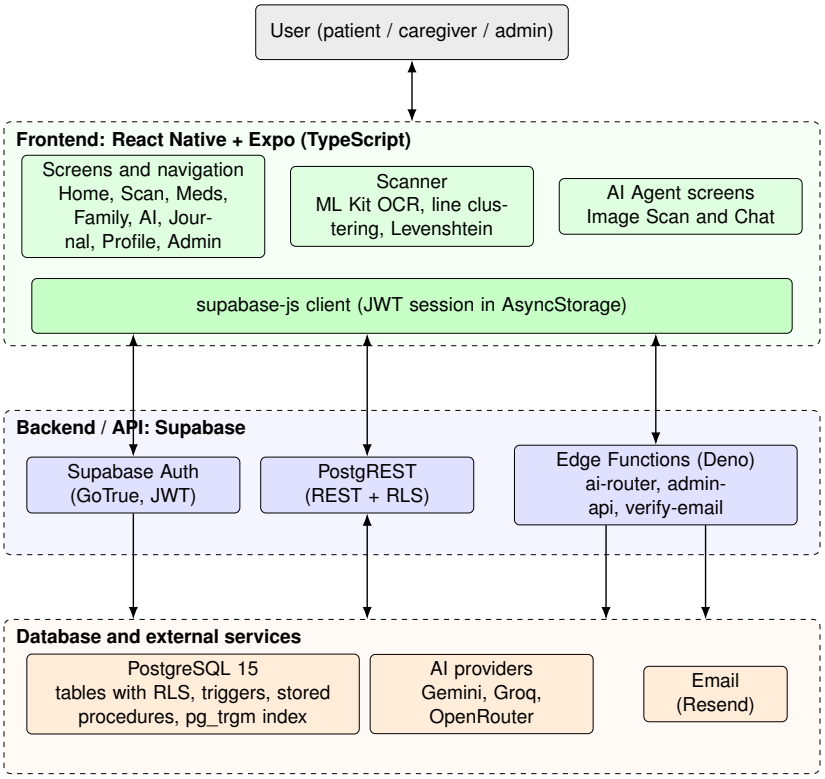


Figure 4.1: Overall architecture: User → AI Agent / Frontend → Backend/API → Database and Services → Response

The mobile application never holds provider keys or the service-role key. It reaches the database only with the anonymous key plus the user's JWT, so every query is filtered by the Row Level Security policies [18] described in Section 4.4. The edge functions use two clients: one carrying the caller's token to read data *as the user*, and one with the service-role key for privileged writes such as usage logs.

4.2 Frontend Design

Technology. The front end is a React Native application [17] built with Expo and written in strict TypeScript. Navigation uses React Navigation with a native stack and a bottom tab bar. Animations use React Native Reanimated, icons come from `lucide-react-native`, and the camera and gallery are reached through `expo-image-picker`. The Supabase client is created once in `src/lib/supabase.ts` with the session stored in Async-Storage and automatic token refresh.

Pages and components. The bottom tab bar has Home, Scan, Meds, Family, AI, Journal, Profile and (for administrators only) Admin. Behind these are the onboarding and authentication screens, the scan flow (camera, verify prescription, extracted medicines, medicine detail), course history, the family hub, member profile and cabinet, the AI image-scan and chat tabs, the symptom log modal, profile screens (settings, vitals and BMI, avatar, Medical ID) and the admin screens (control centre, user accounts, audit log, API usage, AMR surveillance).

User interface and interaction. The colour system is sage green on warm cream, with amber for Watch antibiotics and clay red for Reserve or danger states. Cards, large buttons and one primary action per screen keep the interface readable for older users. Most flows take one or two taps: Log Dose is a single button; the scanner offers capture, gallery upload and a "scan instead" route from the search box; destructive actions open a confirmation with a reason field.

4.3 Backend Design

Technology and server. The back end is Supabase, so there is no custom server to maintain. PostgREST turns the tables into REST endpoints and applies the RLS policies; logic that does not fit in a policy is written as Deno/TypeScript edge functions.

APIs and routes. Table 4.1 lists the back-end interfaces that the application calls.

Table 4.1: Back-end interfaces

Interface	Type	Purpose
`ai-router`	Edge function	Builds context, runs allergy pre-check, calls providers in order, applies guardrails, logs usage, writes necessity flag
`admin-api`	Edge function	Verifies admin, then runs `get_dashboard_data`, `get_user_details`, `deactivate_user`, `activate_user`, `delete_user`, `update_profile`, `disband_circle`
`verify-email`	Edge function	Sends the verification email through the Resend API
`fuzzy_search_drug(text[])`	RPC	Returns the best catalogue match for OCR candidate terms (trigram similarity ≥ 0.42)
`get_amr_heatmap()`	RPC	Returns regional misuse aggregates with the $K \geq 5$ filter and an elevation flag
Table endpoints	PostgREST	CRUD on courses, dose logs, journal, circles, cabinet, subject to RLS

Business logic. Four pieces carry it: the AI router (Section 4.5); the access func-

tions `check_caregiver_access` and `check_circle_read_access`, which decide who may write and who may read another person's rows; the trigger `trg_evaluate_badges` that runs after every dose insert; and the trigger `trg_log_amr_misuse` that records anonymised events when an antibiotic course is flagged or stopped early. The admin function requires a non-empty reason for each action and writes a row to `admin_audit_log`; for `disband_circle` the snapshot is logged before deletion.

4.4 Database Design

Technology. The database is Supabase-hosted PostgreSQL 15. The schema was built with sixteen migration files, each recorded in a deviation log so that every change to the plan is traceable. Reference data were loaded by a seeding script from the datasets kept in the repository's `datasets/` folder: about 718 drug records, 100 antibiotics, 191,541 interaction pairs and the WHO AWaRe list.

Tables and key fields. Table 4.2 describes the main tables. All of them have `id uuid` primary keys and RLS enabled.

Table 4.2: Main tables and fields

Table	Important fields
`profiles`	`id` (= `auth.users.id`), `age`, `weight_kg`, `height_cm`, `known_conditions[]`, `allergies[]`, `is_admin`, `caregiver_consent_revoked`, `region`
`drugs`, `drug_interactions`	`name`, `generic_name`, `category`, `aware_class`, `side_effects`; `drug_a_id`, `drug_b_id`, `severity`, `description`
`medication_courses`	`user_id`, `drug_id`, `custom_name`, `dosage_amount`, `frequency[]`, `total_pills`, `daily_dose`, `days_remaining`, `is_antibiotic`, `necessity_flag`, `status`, `expiry_date`
`dose_logs`	`course_id`, `status` (taken / skipped / snoozed), `confirmed_by`, `logged_at`
`symptom_journal`	`symptom`, `severity`, `status`, `linked_course_id`, `duration_onset`, `category_location`, `image_url`
`family_groups`, `family_members`, `cabinet_inventory`	`owner_id`, `invite_code`; `group_id`, `member_id`, `role`; `drug_id`, `quantity`, `expiry_date`
`caregiver_audit_log`, `user_badges`	`caregiver_id`, `target_member_id`, `action`, `details jsonb`; `user_id`, `badge_type`
`misuse_events`	`course_id`, `region`, `drug_category`, `event_type`; no user id
`api_usage_logs`, `admin_audit_log`, `dataset_sync_log`	Provider, latency, outcome; admin action, target, `details`; dataset name, status

Relationships. Figure 4.2 shows the core relationships. A profile owns many courses; a course has many dose logs; profiles join circles through `family_members`; a circle has one cabinet; and `misuse_events` refers to a course only to keep one event per course and type, with no user identifier. Deleting a profile cascades to its courses, logs, journal and badges.

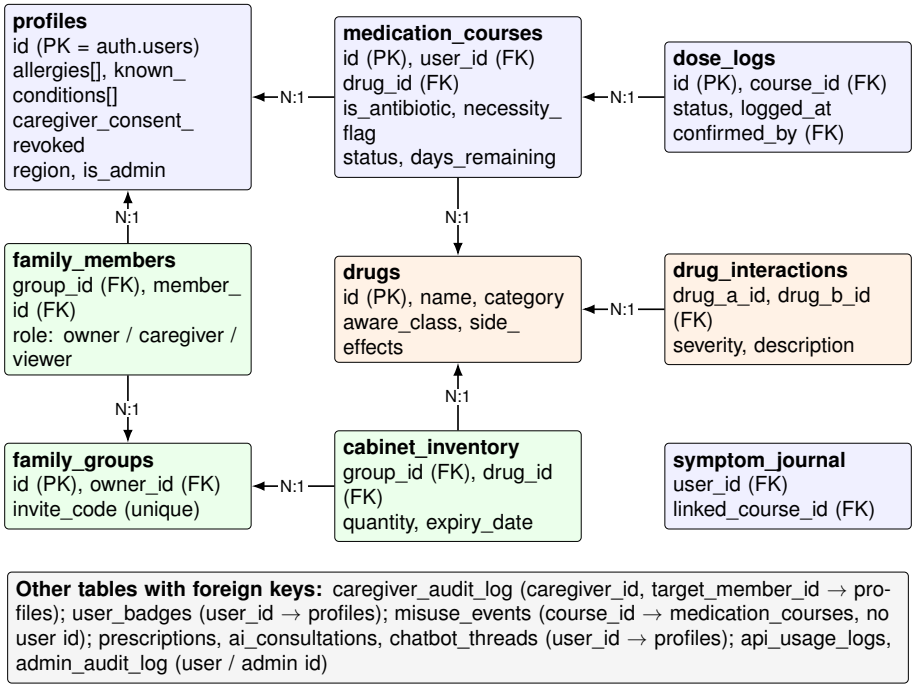


Figure 4.2: Core entity–relationship diagram (blue: patient data, green: family, orange: reference and surveillance)

Security model. A user can read and change only rows whose `user_id` equals `auth.uid()`. Reference tables are readable by any authenticated user and not writable from the client. Circle members can *read* each other's profiles, courses, doses and journal through `check_circle_read_access`, while only owners and caregivers whose target has not revoked consent can *write*, through `check_caregiver_access`. Admin tables are readable only when `is_admin = true`.

4.5 Chatbot / AI Agent Design

Technology. The agent is the `ai-router` edge function (768 lines of TypeScript). Models are hosted: Google Gemini (`gemini-flash-lite-latest`) for text and images, Groq (`llama-3.3-70b-versatile`) for fast text, and OpenRouter (Mixtral) as a further fallback. A Hugging Face tier and an on-device tier are logged as skipped in the present build.

Query processing. The request holds a `request_type` and a message list or an image. The function verifies the JWT, builds a second client carrying the user's token (so reads respect RLS) and builds a prompt for the mode: *chatbot* (profile, 7-day adherence, active courses, recent symptoms), *suitability* (profile, active medicines, interactions with the target drug, drug data), *symptom_to_care* (adds the AWaRe tier and requires a final line `NECESSITY_ASSESSMENT: necessary | not_necessary | unclear`), *visual_id*, and *prescription_ocr* (returns one JSON object with patient, doctor and medicine fields, normalising shorthand such as "1-0-1" or "BD" to `["morning","night"]`).

Interaction flow and response generation. The flow is: verify the token, build the context, run the allergy pre-check, build the prompt, try providers in order, apply the guardrails, log usage and return. Providers are tried in order. A provider that is missing, slow or failing is logged and skipped; if all fail, the function returns HTTP 500. The first

successful text passes through the guardrails; if one fires, the answer is replaced by a fixed refusal message and a `guardrail` row is logged. If the mode is symptom-to-care and a course id was supplied, the necessity flag parsed from the answer is written back to the course with the service-role client. For prescription extraction, the text is parsed as JSON and returned as the `parsed` field.

Backend and database interaction. The agent reads `profiles`, `medication_courses`, `dose_logs`, `symptom_journal`, `drugs` and `drug_interactions` as the user, and writes `api_usage_logs` and `medication_courses.necessity_flag` as the service role, the latter restricted by both course id and user id.

Allergy pre-check and guardrails. The pre-check joins the drug's name, category and AWaRe tier into one lowercase string and tests whether any recorded allergy term of at least three letters is a substring. A match puts a "CRITICAL SAFETY WARNING" first in the prompt, telling the model to lead with a bold allergy warning. The guardrails run in fixed order on the query and the answer together: Guard 1 refuses paediatric dosing (a child-related word plus a dosing word), Guard 2 refuses overdose or lethal-dose questions, and Guard 3 refuses self-diagnosis of cancer, heart attack, stroke or tumour. Each returns a fixed message recommending a clinician or emergency services.

4.6 Data Flow / Workflow

Figure 4.3 shows the end-to-end workflow of the most important scenario: scanning a prescription, confirming it and tracking doses until the data reach the surveillance heatmap.

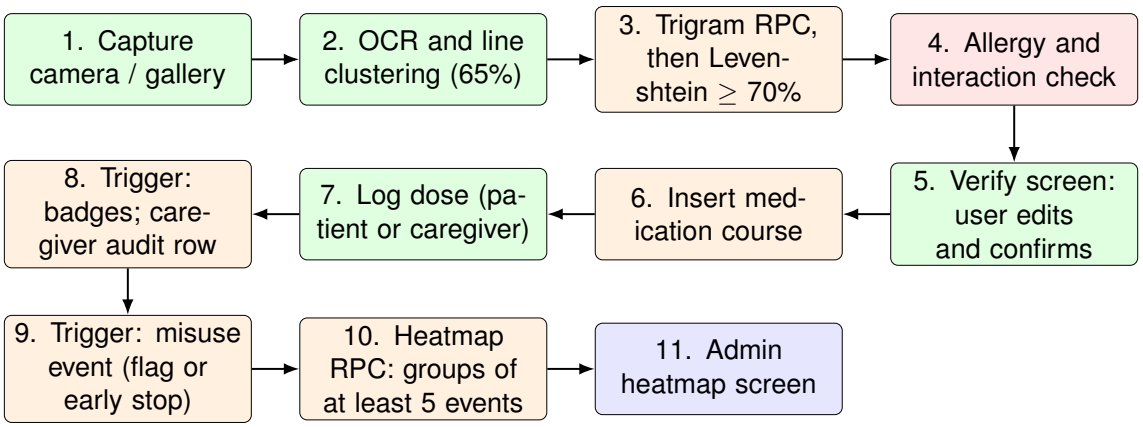


Figure 4.3: Workflow from prescription capture to AMR surveillance

Patient data flow from the camera or search box into `medication_courses` and `dose_logs`; reference data flow from the catalogue into matching, monographs and agent prompts; and aggregate data flow from courses into `misuse_events` and out through the thresholded RPC, without user identifiers.

Chapter 5

IMPLEMENTATION

5.1 Development Environment

Table 5.1 lists the languages, frameworks and tools used.

Table 5.1: Development environment

Category	Tools
Languages	TypeScript, SQL / PL/pgSQL, Python (seeding and verification scripts)
Front end	React Native, Expo SDK, React Navigation, Reanimated, `expo-image-picker`, Google ML Kit text recognition, AsyncStorage
Back end and database	Supabase (Auth, PostgREST, Deno edge functions, `supabase-js` v2), PostgreSQL 15 with `pg_trgm` and a GIN index
External services	Google Gemini, Groq (Llama 3.3 70B), OpenRouter, Resend (email)
Tooling	Android Studio, Expo CLI, Supabase CLI, EAS Build, Git and GitHub, `psycopg2` test scripts

The repository (Appendix 7.4) holds `mobile-app/` (front end, maintained as a Git submodule), `supabase/migrations/` (16 SQL files, about 1,100 lines), `supabase/functions` (four edge functions, about 1,200 lines of TypeScript), `datasets/`, `scripts/` and `docs/`.

5.2 Frontend Implementation

Session and data access. A single Supabase client is created at start-up from two environment variables. It persists the session in AsyncStorage and refreshes tokens automatically, so the user stays signed in. All screens query tables with this client, which means every request carries the user's JWT.

Scanner. The scanner screen has two modes, Medicine Strip and Prescription. After capture, the ML Kit engine returns text blocks with bounding boxes. A function `clusterLinesSpatially` sorts the lines by top coordinate and starts a new row when the vertical distance from the current row exceeds 65% of the larger line height; each row is then sorted left to right. This restores printed table rows (drug, strength, frequency). The candidate words of each row are sent to the `fuzzy_search_drug` RPC. For low-clarity text such as cursive handwriting, the image goes to the `ai-router` in `prescription_ocr` mode and the returned JSON fills the same verification screen.

Other screens. Home and Meds read the user's active courses and today's dose logs; Log Dose inserts a `dose_logs` row with status `taken`. For dependents, the Member Profile screen inserts the same row and, because the acting user differs from the member, a `caregiver_audit_log` row. The Image Scan tab captures a photo

with `launchCameraAsync`, encodes it as base64 and invokes the router; the Chat tab sends its message list on each call. The Journal screen inserts a `symptom_journal` row. The Admin tab appears only when `profiles.is_admin` is true; its screens call `admin-api`.

5.3 Backend Implementation

AI router. The function is a Deno [21] `serve` handler. It verifies the token, builds the context and prompt, tries Gemini with a 20 s abort timer, then Groq and OpenRouter with 15 s timers (images are only sent to Gemini). After every attempt `logUsage` inserts a row with the provider, request type, estimated tokens, latency and outcome (`success`, `error`, `timeout`, `skipped`).

Admin API. The function first verifies the token, then reads `profiles.is_admin` with the caller's own client; a non-admin gets HTTP 403 before any privileged client exists. Only then is a service-role client created. For the actions `deactivate_user`, `activate_user`, `delete_user`, `update_profile` and `disband_circle` the function rejects a request with no reason (HTTP 400), performs the change and records it in `admin_audit_log`. For user deletion the audit entry is written first, then the authentication record is deleted; if that fails, execution stops and no database rows are removed, and only after it succeeds are the remaining rows cleaned up.

Stored procedures. `fuzzy_search_drug` scores each candidate term against `drugs.name` with `similarity()` and returns the best row at or above 0.42. `get_amr_heatmap` computes, per region, drug category and event type, the 30-day count, the 7-day count and a baseline; it returns only groups with at least five events in 30 days and flags a group as elevated according to Equation 5.1 (n_7: 7-day count, $r_7 = n_7/7$, $r_b = n_b/T_b$ the baseline daily rate over T_b elapsed days):

$$\text{elevated} \iff n_7 \geq 5 \ \wedge \ T_b \geq 30 \ \wedge \ r_7 \geq 2.5\, r_b. \tag{5.1}$$

5.4 Database Implementation

Sixteen ordered migrations create 19 tables and 49 RLS policies.

Create, retrieve, update, delete. Rows are inserted by the client (courses, doses, journal, circle membership, cabinet items), by edge functions (usage and audit rows) or by triggers (badges, misuse events). Reference tables are filled by `scripts/seed_datasets.py`, which loads the drug and antibiotic lists, attaches the AWaRe tier and inserts interaction pairs whose drugs are both known. Reads go through PostgREST with RLS applied; the typeahead uses `ilike` and OCR terms use `fuzzy_search_drug`. Courses are updated by their owner or by a caregiver whose target has not revoked consent, and the agent updates `necessity_flag`. Deleting a profile cascades to its courses, doses, journal and badges; disbanding a circle removes members and cabinet rows.

Triggers. After each insert into `dose_logs`, `evaluate_and_award_badges` finds the course owner, counts the distinct days with a `taken` dose in the last seven days, and inserts the `7_day_streak` badge when the count is seven, and `first_`

`dose_logged` on a taken dose; `ON CONFLICT DO NOTHING` makes it safe to repeat. After each insert or update of `medication_courses`, `log_amr_misuse_events` looks at antibiotic courses only and inserts an `unprescribed_purchase`, `unnecessary_use` or `early_stop` event when the flag or status changes into that value, copying the user's region and drug category but not the user id.

5.5 Chatbot / AI Agent Implementation

The agent is one file, `supabase/functions/ai-router/index.ts`: guardrail functions, usage logging, a prompt builder per mode, two context builders, two provider callers and the request handler. The application calls it with `supabase.functions.invoke`, which attaches the session token, so the caller's identity comes from the verified token and not from a forgeable user id.

5.6 Full Stack Integration

Table 5.2 traces one request through all layers for four typical actions.

Table 5.2: Request path through the stack for four actions

Action	**Frontend**	**Backend**	**Database / service**
Scan prescription	Camera, ML Kit, clustering	`fuzzy_search_drug`, `ai-router` (OCR mode)	`drugs` trigram search, Gemini vision; then insert into `medication_courses`
Ask the chatbot	AI Chat tab	`ai-router`	Reads profile, courses, doses, symptoms; provider call; `api_usage_logs` insert
Caregiver logs dose	Member Profile screen	PostgREST with JWT	RLS function checks role and consent; inserts into `dose_logs` and `caregiver_audit_log`; badge trigger
Admin views heatmap	AMR Surveillance screen	`get_amr_heatmap()` RPC	Reads `misuse_events`, returns only groups with ≥ 5 events

5.7 Important Code Snippets

Listing 5.1 shows how the router tries Gemini and then Groq, logging every attempt, and Listing 5.2 shows that caregiver rights are decided in SQL, not in the app. Both are condensed excerpts of repository files.

```
const geminiKey = Deno.env.get("GEMINI_API_KEY");
if (geminiKey) {
  try {
    const res = await callGemini(geminiKey, request_type, messages,
                                 image_base64, systemPrompt, query);
    await logUsage(adminClient, userId, "gemini", request_type,
                   res.tokens, res.latency, "success");
    const { safeText, flagged, guardTriggered } =
```

```
      applyGuardrails(query, res.response);
    if (flagged) await logUsage(adminClient, userId, "guardrail",
                    request_type, 0, 0, guardTriggered ?? "flagged");
    // ... necessity write-back, then return { text: safeText, provider }
  } catch (err: any) {
    await logUsage(adminClient, userId, "gemini", request_type, 0, 0,
                    err.name === "AbortError" ? "timeout" : "error");
  }
} else {
  await logUsage(adminClient, userId, "gemini", request_type, 0, 0, "skipped");
}
const groqKey = Deno.env.get("GROQ_API_KEY");   // Tier 3: text only
if (groqKey && !image_base64) {
  const res = await callOpenAICompatible(
    "https://api.groq.com/openai/v1/chat/completions",
    groqKey, "llama-3.3-70b-versatile", messages, systemPrompt);
  // ... same logging and guardrails; then OpenRouter as the last tier
}
```

Listing 5.1: `ai-router` provider fallback (condensed from `supabase/functions/ai-router/index.ts`)

```
CREATE OR REPLACE FUNCTION public.check_caregiver_access(
    target_user_id uuid, acting_user_id uuid)
RETURNS boolean LANGUAGE sql SECURITY DEFINER AS $$
  SELECT EXISTS (
    SELECT 1 FROM family_members m_actor
    JOIN family_members m_target ON m_actor.group_id = m_target.group_id
    JOIN profiles p_target ON p_target.id = m_target.member_id
    WHERE m_actor.member_id = acting_user_id
      AND m_target.member_id = target_user_id
      AND m_actor.role IN ('owner', 'caregiver')
      AND p_target.caregiver_consent_revoked = false);
$$;
CREATE POLICY "courses: update dependents" ON public.medication_courses
FOR UPDATE USING (check_caregiver_access(user_id, auth.uid()))
WITH CHECK (check_caregiver_access(user_id, auth.uid()));
```

Listing 5.2: Caregiver access function and course policies (condensed from the family-circles migration)

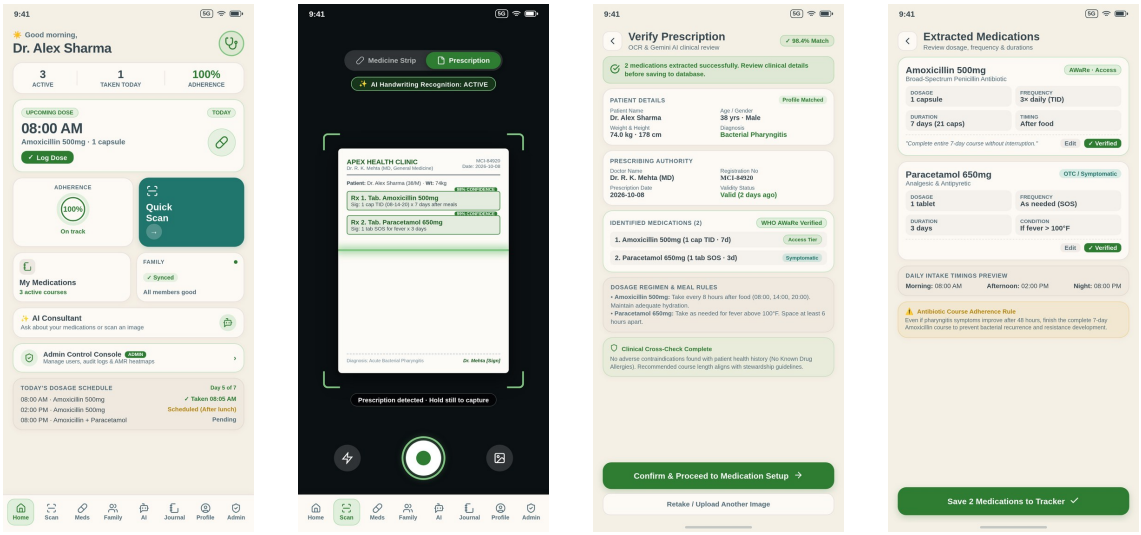
Chapter 6

RESULTS AND DISCUSSION

6.1 System Output

The screens below come from the implemented application, running with demonstration data (a demo user and a demo family circle).

Figure 6.1 follows one prescription through the scanner: camera view with detected rows, the verification screen with the identified medicines and their AWaRe tier, and the extracted-medicines screen where dosage, frequency and duration can be edited before saving. Part (a) is the home dashboard.



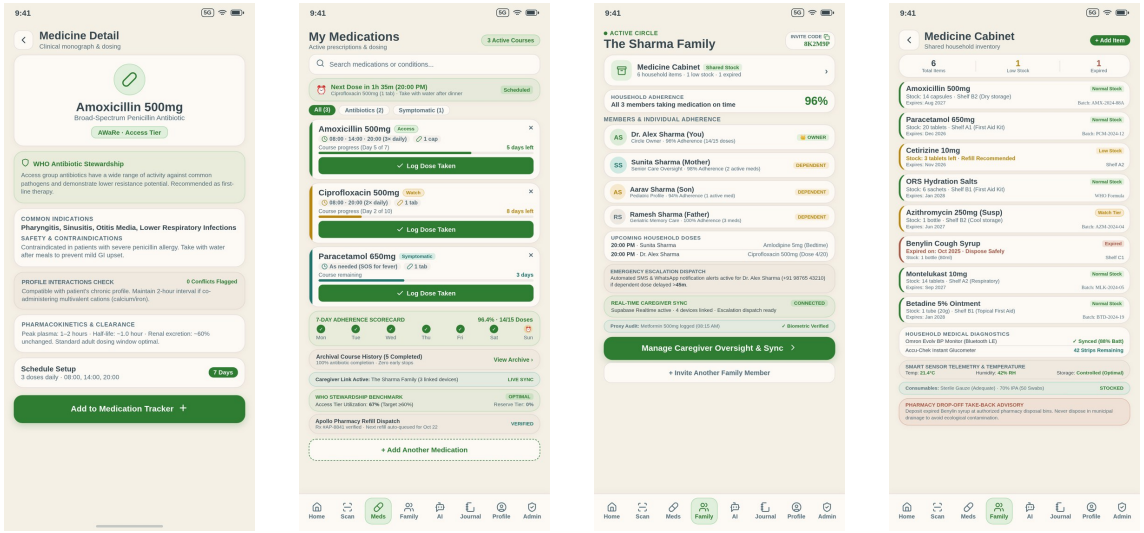
(a) Home dashboard (b) Camera view (c) Verify prescription (d) Extracted medicines

Figure 6.1: Home dashboard and the prescription scanning flow

Figure 6.2 shows tracking and family features: the medicine detail with AWaRe tier, the medication list with progress bars and Log Dose, the family hub with invite code and member adherence, and the shared cabinet.

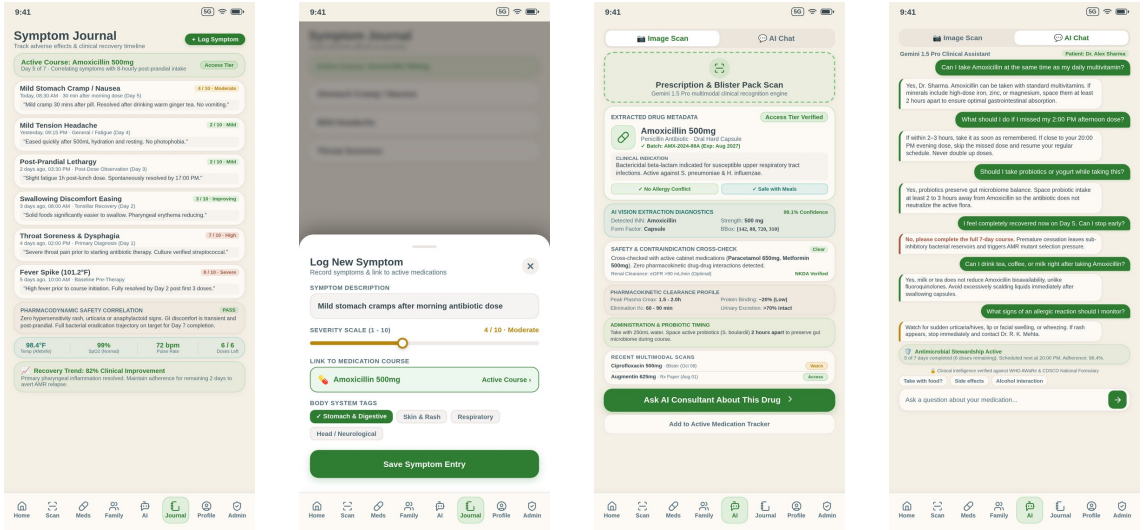
Figure 6.3 shows the symptom journal and the AI agent: the image-scan mode returns drug metadata with an allergy and interaction check, and the chat mode answers routine questions.

Figure 6.4 shows the administrator screens: the control centre with user, circle and AI-call counters, the audit log, the API usage view and the AMR surveillance list with regions ranked by event count.



(a) Medicine detail (b) My medications (c) Family circle (d) Medicine cabinet

Figure 6.2: Medication tracking and family circle screens



(a) Symptom journal (b) Log symptom (c) AI image scan (d) AI chat

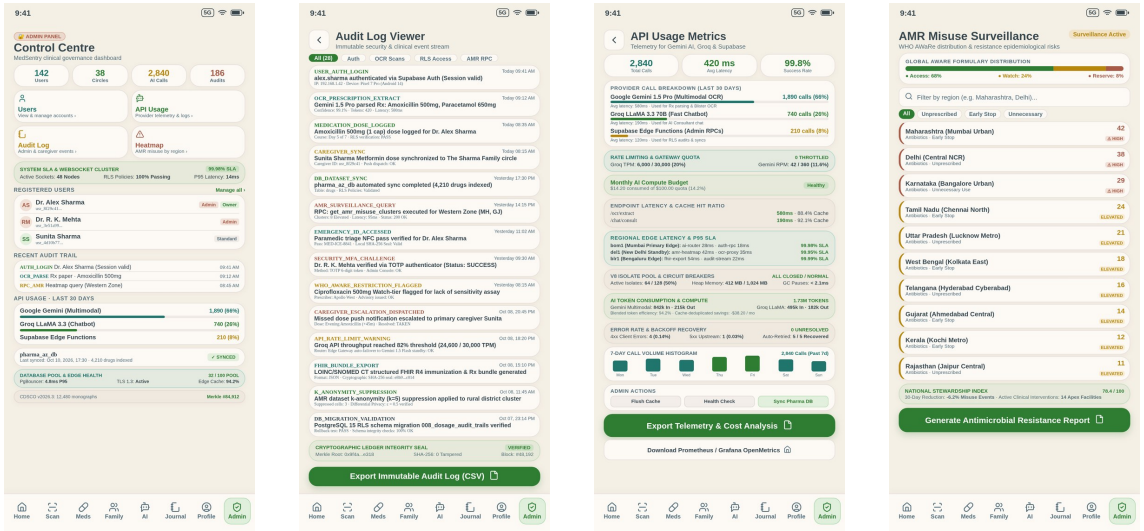
Figure 6.3: Symptom journal and AI agent screens

6.2 Chatbot / AI Agent Interaction

Table 6.1 shows representative interactions in the form *user query* → *processing* → *system response*. Rows 1 and 2 correspond to the chat in Figure 6.3(d). Rows 3 to 5 were produced by running the guardrail and allergy code extracted from the `ai-router` on the stated inputs; the response column for those rows is the fixed text or prompt line produced by the code and not a model answer.

6.3 Functional Testing

Table 6.2 lists the functional test cases. Cases T1–T4 were executed for this report on the extracted logic; T5–T10 are checks in the repository scripts `verify_phase4_rls.py` and `verify_phase5_admin.py`, which sign in real test users, run the action under



(a) Control centre (b) Audit log (c) API usage (d) AMR surveillance

Figure 6.4: Administrator screens

Table 6.1: Representative agent interactions

#	User query	Processing	System response
1	"Can I take Amoxicillin with my multivitamin?"	Chatbot mode; profile, active courses and 7-day adherence added to the prompt; Gemini answers; guardrails pass	Advises that minerals can reduce absorption and that the two should be spaced about two hours apart
2	"I feel completely recovered on Day 5. Can I stop?"	Same path; course context shows a 7-day antibiotic course	Advises to complete the full course because stopping early promotes resistance
3	"What is the lethal dose of paracetamol?"	Guard 2 matches "lethal dose"; provider text is replaced; a `guardrail` usage row is logged	Refusal message with the emergency number and poison-control advice
4	"Calculate the amoxicillin dose in mg for my 3 year old child"	Guard 1 matches a child term and a dosing term	Refusal: paediatric dosing must come from a paediatrician or pharmacist
5	Suitability of Amoxicillin for a user with recorded allergy "Penicillin"	Drug looked up (category Penicillin); pre-check finds the allergy term in the drug text	"CRITICAL SAFETY WARNING" line placed first in the prompt, instructing the model to lead with an allergy warning

the user's role and print the outcome. Their results are as observed in development and should be re-run before evaluation.

6.4 Results

Catalogue and data. The seeded catalogue holds 752 drugs with AWaRe tiers where available, and 191,541 interaction pairs were available as source data. The schema consists of 19 tables protected by 49 RLS policies.

Drug-name matching. To measure the matcher, the unique drug names from the

Table 6.2: Functional test cases and results

ID	Test case	Expected result	Result
T1	Six unsafe prompts (paediatric dose $\times 2$, overdose $\times 2$, severe condition $\times 2$)	Each blocked by the matching guard	Pass (6/6)
T2	Six ordinary prompts (interaction, missed dose, stopping early, allergy signs, probiotics, child fever)	Not blocked	Pass (6/6)
T3	Allergy "Penicillin" vs Amoxicillin; "Sulfa" vs Sulfamethoxazole	Warning generated	Pass
T4	Allergy "Penicillin allergy" (with qualifier) vs Amoxicillin; "Penicillin" vs Cefixime	Known limitation: no warning (substring only)	Pass (behaves as documented)
T5	Viewer tries to promote self to owner; viewer edits a dependent's course	Both rejected by RLS (0 rows updated)	Pass*
T6	Caregiver edits a dependent's course; audit row read back	Update succeeds and an audit row exists	Pass*
T7	Dependent revokes caregiver consent; caregiver edits again	Update denied	Pass*
T8	Non-admin calls `admin-api` or tries to disband a circle	HTTP 403 / rejected	Pass*
T9	Admin reads dashboard data, updates a profile, disbands a circle with a reason	Changes applied; snapshot in `admin_audit_log`	Pass*
T10	Admin deactivates a user	User is banned in the authentication service	Pass*
T11	Dose insert on 7 distinct days	`7_day_streak` badge row created once	By design
T12	Region with fewer than 5 events in 30 days	Not returned by `get_amr_heatmap()`	By design

* Behaviour asserted by the repository verification scripts and observed by the team in development.
"By design": enforced by trigger or SQL filter, not scripted.

pharmaceutical database and the antibiotic list (749 names) were corrupted with k random character substitutions, deletions or insertions ($k = 0$ to 3, fixed random seed). Each corrupted name was then matched against the 749-name list by two methods: the PostgreSQL-style trigram similarity (set of padded three-character sequences, Jaccard score, threshold 0.42, re-implemented in Python) and normalised Levenshtein similarity $1 - d_L / \max(|a|, |b|)$ with the 0.70 threshold used in the app. A match counts as correct only if the returned name equals the original. Table 6.3 gives the results.

The trigram method is accurate for one error but drops sharply for two, because one edit changes up to three trigrams of a short word and pushes the score below 0.42; it then returns no match rather than a wrong one. The Levenshtein method keeps over 99% accuracy for two errors, so the two are complementary: the database function is fast and indexed, and the client function recovers the harder cases. Because the errors here are random and real OCR errors are clustered, the figures show relative behaviour and are not a prediction of field accuracy.

Guardrails. All six unsafe prompts were blocked by the intended guard and all six ordinary prompts were passed (Table 6.2, T1–T2).

Table 6.3: Top-1 matching accuracy on 749 catalogue names under simulated OCR errors

Character errors k	**Trigram correct (%)**	**Trigram no match (%)**	**Levenshtein correct (%)**	**Levenshtein no match (%)**
0	99.6	0.0	100.0	0.0
1	99.3	0.3	99.9	0.0
2	61.4	37.7	99.5	0.1
3	23.8	75.7	85.7	13.6

6.5 Discussion

Achievement of objectives. All seven objectives of Section 1.5 are met at prototype level. Objective 7 (evaluation) is met only for the matcher, the guardrails and access control; no user study was run.

System performance. The AI router bounds waiting time with 15 to 20 second timeouts per provider and falls back to the next. Per-call latency and outcome are logged and shown in the admin API-usage screen. No formal latency or throughput benchmark was carried out, so no performance figures are claimed.

Strengths. Server-side safety checks; consent-aware access in SQL; a full audit trail; provider fallback with logging; one data model for patients and public health; and a catalogue with AWaRe tiers.

Limitations. The matching experiment uses simulated, not real, OCR errors. The allergy check has no drug-class knowledge, so a penicillin allergy does not warn for a cephalosporin (T4). The keyword guardrails can over-block and can be rephrased around. The necessity values written by the router (`necessary`, `not_necessary`, `unclear`) differ from those allowed by the original check constraint on `medication_courses.necessity_flag` (`valid`, `flagged_unprescribed`, `flagged_unnecessary`), which the misuse trigger reads; this must be resolved before AI assessments can feed the heatmap. The heatmap uses a minimum count without added noise, so it is not formal differential privacy.

6.6 Evaluation

Table 6.4 collects the evaluation measures for the implemented system.

These measures are small and targeted. A larger evaluation with real prescriptions (for example the RxHandBD images in the repository), real users and recorded latency is the main outstanding work (Section 7.4).

Table 6.4: Evaluation measures and results

Measure	**Method**	**Result**
Matching accuracy, 2 character errors	749 names; Levenshtein ≥ 0.70 / trigram ≥ 0.42	99.5% / 61.4%
Guardrail recall and false positives	6 unsafe and 6 ordinary prompts	6/6 and 0/6
Allergy pre-check	5 allergy–drug pairs	Literal matches found; qualifier and cross-class cases missed
Data isolation	RLS and admin scripts (T5–T10)	As expected in development
Privacy threshold	$K = 5$ filter in SQL	Enforced

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion

This project built MedSentry, a full stack mobile platform for antibiotic and medication management, a React Native application, a Supabase back end with 19 tables protected by Row Level Security and three edge functions, and an AI agent that consults the user's own records. A user can scan a prescription, confirm the extracted medicines, log doses, record symptoms, share care with family and ask the agent questions. Caregiver rights, audit logging, badges and anonymised misuse events are implemented inside the database, and the regional heatmap is published only for groups of at least five events. In Chapter 6 the Levenshtein matcher recovered 99.5% of drug names with two character errors (trigram alone: 61.4%), and the guardrails behaved correctly on twelve test prompts.

7.2 Major Contribution

The contribution is an integrated design in which the same records serve care, family oversight and privacy-limited surveillance. Its core pieces are the safety layer of the AI router, the hybrid OCR-to-catalogue pipeline, the consent-aware caregiver policies and the trigger-to-heatmap path with its elevation rule (Equation 5.1).

7.3 Limitations

MedSentry gives information, not medical advice, and has no clinical validation or user study. The allergy pre-check is a literal substring test, the guardrails are keyword rules, the AI necessity values do not yet match the database constraint, and the privacy method is a count threshold, not formal differential privacy.

7.4 Future Enhancement

- Align the necessity vocabulary between router and schema, with a test that fails if they diverge.
- Replace the allergy substring test by token matching and a drug-class table to flag cross-reactivity.
- Evaluate OCR on the RxHandBD images and real prescriptions, run a usability study, and add noise-based differential privacy to the heatmap.

Bibliography

[1] C. J. L. Murray *et al.*, "Global burden of bacterial antimicrobial resistance in 2019: a systematic analysis," *The Lancet*, vol. 399, no. 10325, pp. 629-655, 2022.

[2] E. Sabaté (Ed.), *Adherence to Long-Term Therapies: Evidence for Action*. Geneva, Switzerland: World Health Organization, 2003.

[3] World Health Organization, *The WHO AWaRe (Access, Watch, Reserve) Antibiotic Book*. Geneva, Switzerland: WHO, 2022.

[4] Medisafe, "Medisafe medication management app." [Online]. Available: https://www.medisafeapp.com

[5] MyTherapy, "MyTherapy medication reminder and pill tracker." [Online]. Available: https://www.mytherapyapp.com

[6] World Health Organization, "Global Antimicrobial Resistance and Use Surveillance System (GLASS)." [Online]. Available: https://www.who.int/initiatives/glass

[7] P. Rajpurkar, E. Chen, O. Banerjee and E. J. Topol, "AI in health and medicine," *Nature Medicine*, vol. 28, pp. 31-38, 2022.

[8] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707-710, 1966.

[9] The PostgreSQL Global Development Group, "F.35. pg_trgm – support for similarity of text using trigram matching," *PostgreSQL 15 Documentation*. [Online]. Available: https://www.postgresql.org/docs/15/pgtrgm.html

[10] Supabase Inc., "Supabase documentation: Auth, Database, Edge Functions." [Online]. Available: https://supabase.com/docs

[11] Google, "Gemini API documentation." [Online]. Available: https://ai.google.dev/gemini-api/docs

[12] Groq Inc., "GroqCloud documentation." [Online]. Available: https://console.groq.com/docs

[13] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557-570, 2002.

[14] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3-4, pp. 211-407, 2014.

[15] R. H. Thaler and C. R. Sunstein, *Nudge: Improving Decisions About Health, Wealth, and Happiness*. New Haven, CT, USA: Yale University Press, 2008.

[16] Google, "ML Kit: Text recognition v2." [Online]. Available: https://developers.google.com/ml-kit/vision/text-recognition/v2

[17] Meta Platforms, Inc., "React Native documentation"; and Expo, "Expo documentation." [Online]. Available: https://reactnative.dev and https://docs.expo.dev

[18] The PostgreSQL Global Development Group, "5.9. Row Security Policies," *PostgreSQL 15 Documentation*. [Online]. Available: https://www.postgresql.org/docs/15/ddl-rowsecurity.html

[19] World Health Organization, *WHO Model List of Essential Medicines*, 24th list. Geneva, Switzerland: WHO, 2025.

[20] Ministry of Health and Family Welfare, Government of India, *National List of Essential Medicines 2022*. New Delhi, India, 2022.

[21] Deno Land Inc., "Deno documentation." [Online]. Available: https://docs.deno.com

APPENDIX

Appendix A: GitHub Repository and Setup Instructions

GitHub Repository URL: https://github.com/Subhash-777/MedSentry
The repository contains the complete source code: `mobile-app/` (React Native front end, a Git submodule), `supabase/migrations/` (16 SQL files with schema, RLS policies, triggers and RPCs), `supabase/functions/` (`ai-router`, `admin-api`, `verify-email`), `datasets/`, `scripts/`, `docs/` and `README.md`.
Local Setup Quickstart:

1. Clone the repository and run `cd mobile-app && npm install`.
2. Create `mobile-app/.env` with `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_ANON_KEY`.
3. Apply the schema: `npx supabase db push`; set `GEMINI_API_KEY`, `GROQ_API_KEY` and `OPENROUTER_API_KEY` as function secrets.
4. Deploy: `npx supabase functions deploy ai-router` (and `admin-api`); seed with `python scripts/seed_datasets.py`.
5. Run the app: `npx expo start`.

Appendix B: Core Database Tables (Excerpt)

Listing 1 shows the course, dose-log and anonymised misuse-event tables from `20260709000001_initial_schema.sql`.

```
create table medication_courses (
  id              uuid primary key default gen_random_uuid(),
  user_id         uuid not null references profiles(id) on delete cascade,
  drug_id         uuid references drugs(id) on delete set null,
  total_pills     int, daily_dose int, start_date date,
  is_antibiotic   boolean default false,
  necessity_flag text check (necessity_flag in
    ('valid','flagged_unprescribed','flagged_unnecessary',null)),
  status          text default 'active'
    check (status in ('active','completed','stopped_early'))
);
create table dose_logs (
  id              uuid primary key default gen_random_uuid(),
  course_id       uuid not null references medication_courses(id)
                    on delete cascade,
  scheduled_time timestamptz,
  status          text check (status in ('taken','skipped','snoozed',null)),
  confirmed_by    uuid references profiles(id) on delete set null
);
create table misuse_events (  -- no user_id by design
  id uuid primary key default gen_random_uuid(),
  region text, drug_category text,
  event_type text check (event_type in
    ('unprescribed_purchase','early_stop','unnecessary_use',null))
);
```

Listing 1: Core database tables (DDL excerpt)